

Тема 3.3 Перегрузка операторов. Свойства и декораторы классов

Специальные методы

Классы поддерживают следующие специальные методы:

– `__call__()` - позволяет обработать вызов экземпляра класса как вызов функции. **Формат метода:**

```
__call__(self [, <Параметр1>[, ..., <ПараметрN>]])
```

Пример:

```
class MyClass:
```

```
    def __init__(self, m):
```

```
        self.msg = m
```

```
    def __call__(self):
```

```
        print(self.msg)
```

```
c1 = MyClass("Значение1") # Создание экземпляра класса
```

```
c2 = MyClass("Значение2") # Создание экземпляра класса
```

```
c1() # Выведет: Значение1
```

```
c2() # Выведет: Значение2
```

Результат работы приложения:

Значение1

Значение2

– `__getattr__(self, <Атрибут>)` - вызывается при обращении к несуществующему атрибуту класса. **Пример:**

```
class MyClass:
```

```
    def __init__(self):
```

```
        self.i = 20
```

```
    def __getattr__(self, attr):
```

```
        print("Вызван метод __getattr__()")
```

```
        return 0
```

```
c = MyClass()
```

```
# Атрибут i существует
```

```
print(c.i) # Выведет: 20. Метод __getattr__()
```

```
# Атрибут s не существует
```

```
print(c.s) # Выведет: Вызван метод __getattr__()
```

Результат работы приложения:

20

Вызван метод __getattr__()

0

– `__getattribute__(self, <Атрибут>)` - вызывается при обращении к любому атрибуту класса. Необходимо учитывать, что использование точечной нотации (для обращения к атрибуту класса) внутри этого метода приведет к заиклииванию. Чтобы избежать заиклиивания, следует вызывать метод `__getattribute__()` объекта **object**. Внутри метода нужно вернуть значение атрибута или возбудить исключение **AttributeError**. **Пример:**

```
class MyClass:
```

```
    def __init__(self):
```

```
        self.i = 20
```

```
def __getattr__(self, attr):
    print("Вызван метод __getattr__()")
    return object.__getattr__(self, attr) # Только так!!!
c = MyClass()
print(c.i) # Выведет: Вызван метод __getattr__() 20
```

Результат работы приложения:

```
Вызван метод __getattr__()
20
```

– **__setattr__(self, <Атрибут>, <Значение>)** - вызывается при попытке присваивания значения атрибуту экземпляра класса. Если внутри метода необходимо присвоить значение атрибуту, то следует использовать словарь **__dict__**, иначе при точечной нотации метод **__setattr__()** будет вызван повторно, что приведет к заикливанию. **Пример:**

```
class MyClass:
    def __setattr__(self, attr, value):
        print("Вызван метод __setattr__()")
        self.__dict__[attr] = value
c = MyClass()
c.i = 10 # Выведет: Вызван метод __setattr__()
print(c.i) # Выведет: 10
```

Результат работы приложения:

```
Вызван метод __setattr__()
10
```

– **__delattr__(self, <Атрибут>)** - вызывается при удалении атрибута с помощью инструкции **del <Экземпляр класса>.<Атрибут>**;

– **__len__(self)** - вызывается при использовании функции **len()**, а также для проверки объекта на логическое значение при отсутствии метода **__bool__()**. Метод должен возвращать положительное целое число. **Пример:**

```
class MyClass:
    def __len__(self):
        return 50
c = MyClass()
print(len(c)) # Выведет: 50
```

Результат работы приложения:

```
50
```

– **__bool__(self)** - вызывается при использовании функции **bool()**;

– **__int__(self)** - вызывается при преобразовании объекта в целое число с помощью функции **int()**;

– **__float__(self)** - вызывается при преобразовании объекта в вещественное число с помощью функции **float()**;

– **__complex__(self)** - вызывается при преобразовании объекта в комплексное число с помощью функции **complex()**;

– **__round__(self, n)** - вызывается при использовании функции **round()**;

– **__index__(self)** - вызывается при использовании функций **bin()**, **hex()** и **oct()**;

– `__repr__(self)` и `__str__(self)` - служат для преобразования объекта в строку. Метод `__repr__()` вызывается при выводе в интерактивной оболочке, а также при использовании функции `repr()`. Метод `__str__()` вызывается при выводе с помощью функции `print()`, а также при использовании функции `str()`. Если метод `__str__()` отсутствует, то будет вызван метод `__repr__()`. В качестве значения методы `__repr__()` и `__str__()` должны возвращать строку. **Пример:**

```
class MyClass:
    def __init__(self, m):
        self.msg = m
    def __repr__(self):
        return "Вызван метод __repr__() {0}".format(self.msg)
    def __str__(self):
        return "Вызван метод __str__() {0}".format(self.msg)
c = MyClass("Значение")
print(repr(c)) # Выведет: Вызван метод __repr__() Значение
print(str(c)) # Выведет: Вызван метод __str__() Значение
print(c)      # Выведет: Вызван метод __str__() Значение
```

Результат работы приложения:

```
Вызван метод __repr__() Значение
Вызван метод __str__() Значение
Вызван метод __str__() Значение
```

– `__hash__(self)` - этот метод следует переопределить, если экземпляр класса планируется использовать в качестве ключа словаря или внутри множества.

Пример:

```
class MyClass:
    def __init__(self, y):
        self.x = y
    def __hash__(self):
        return hash(self.x)
c = MyClass(10)
d = {}
d[c] = "Значение"
print(d[c]) # Выведет: Значение
```

Результат работы приложения:

```
Значение
```

Классы поддерживают еще несколько специальных методов, которые применяются лишь в особых случаях и будут рассмотрены позднее.

Перегрузка операторов

Перегрузка операторов позволяет экземплярам классов участвовать в обычных операциях. Чтобы перегрузить оператор, необходимо в классе определить метод со специальным названием. Для перегрузки математических операторов используются следующие методы:

– `x + y` - сложение: `x.__add__(y)`;

- $y + x$ - сложение (экземпляр класса справа): `x.__radd__(y)`;
- $x += y$ - сложение и присваивание: `x.__iadd__(y)`;
- $x - y$ - вычитание: `x.__sub__(y)`;
- $y - x$ - вычитание (экземпляр класса справа): `x.__rsub__(y)`;
- $x -= y$ - вычитание и присваивание: `x.__isub__(y)`;
- $x * y$ - умножение: `x.__mul__(y)`;
- $y * x$ - умножение (экземпляр класса справа): `x.__rmul__(y)`;
- $x *= y$ - умножение и присваивание: `x.__imul__(y)`;
- x / y - деление: `x.__truediv__(y)`;
- y / x - деление (экземпляр класса справа): `x.__rtruediv__(y)`;
- $x /= y$ - деление и присваивание: `x.__itruediv__(y)`;
- $x // y$ - деление с округлением вниз: `x.__floordiv__(y)`;
- $y // x$ - деление с округлением вниз (экземпляр класса справа): `x.__rfloordiv__(y)`;
- $x // y$ - деление с округлением вниз и присваивание: `x.__ifloordiv__(y)`;
- $x \% y$ - остаток от деления: `x.__mod__(y)`;
- $y \% x$ - остаток от деления (экземпляр класса справа): `x.__mod__(y)`;
- $x \% = y$ - остаток от деления и присваивание: `x.__imod__(y)`;
- $x ** y$ - возведение в степень: `x.__pow__(y)`;
- $y ** x$ - возведение в степень (экземпляр класса справа): `x.__rpow__(y)`;
- $x ** = y$ - возведение в степень и присваивание: `x.__ipow__(y)`;
- $-x$ - унарный минус: `x.__neg__()`;
- $+x$ - унарный плюс: `x.__pos__()`;
- `abs(x)` - абсолютное значение: `x.__abs__()`.

Пример перегрузки математических операторов:

```
class MyClass:
```

```
    def __init__(self, y):
        self.x = y
    def __add__(self, y): # Перегрузка оператора +
        print("Экземпляр слева")
        return self.x + y
    def __radd__(self, y): # Перегрузка оператора +
        print("Экземпляр справа")
        return self.x + y
    def __iadd__(self, y): # Перегрузка оператора +=
        print("Сложение с присваиванием")
        self.x += y
        return self
```

```
c = MyClass(50)
```

```
print(c + 10) # Выведет: Экземпляр слева 60
```

```
print(20 + c) # Выведет: Экземпляр справа 70
```

```
c += 30 # Выведет: Сложение с присваиванием
```

```
print(c.x) # Выведет: 80
```

Результат работы приложения:

Экземпляр слева

60

Экземпляр справа

70

Сложение с присваиванием

80

Методы перегрузки двоичных операторов:

- $\sim x$ - двоичная инверсия: `x.__invert__()`;
- $x \& y$ - двоичное И: `x.__and__(y)`;
- $y \& x$ - двоичное И (экземпляр класса справа): `x.__rand__(y)`;
- $x \&= y$ - двоичное И и присваивание: `x.__iand__(y)`;
- $x | y$ - двоичное ИЛИ: `x.__or__(y)`;
- $y | x$ - двоичное ИЛИ (экземпляр класса справа): `x.__ror__(y)`;
- $x |= y$ - двоичное ИЛИ и присваивание: `x.__ior__(y)`;
- $x \wedge y$ - двоичное исключающее ИЛИ: `x.__xor__(y)`;
- $y \wedge x$ - двоичное исключающее ИЛИ (экземпляр класса справа):

`x.__rxor__(y)`;

- $x \wedge= y$ - двоичное исключающее ИЛИ и присваивание: `x.__ixor__(y)`;
- $x \ll y$ - сдвиг влево: `x.__lshift__(y)`;
- $y \ll x$ - сдвиг влево (экземпляр класса справа): `x.__rlshift__(y)`;
- $x \ll= y$ - сдвиг влево и присваивание: `x.__ilshift__(y)`;
- $x \gg y$ - сдвиг вправо: `x.__rshift__(y)`;
- $y \gg x$ - сдвиг вправо (экземпляр класса справа): `x.__rrshift__(y)`;
- $x \gg= y$ - сдвиг вправо и присваивание: `x.__irshift__(y)`.

Перегрузка операторов сравнения производится с помощью следующих методов:

- $x == y$ - равно: `x.__eq__(y)`;
- $x != y$ - не равно: `x.__ne__(y)`;
- $x < y$ - меньше: `x.__lt__(y)`;
- $x > y$ - больше: `x.__gt__(y)`;
- $x \leq y$ - меньше или равно: `x.__le__(y)`;
- $x \geq y$ - больше или равно: `x.__ge__(y)`;
- $y \text{ in } x$ - проверка на вхождение: `x.__contains__(y)`.

Приведем пример перегрузки операторов сравнения:

```
class MyClass:
```

```
    def __init__(self):
```

```
        self.x = 50
```

```
        self.arr = [1, 2, 3, 4, 5]
```

```
    def __eq__(self, y):    # Перегрузка оператора ==
```

```
        return self.x == y
```

```
    def __contains__(self, y): # Перегрузка оператора in
```

```
        return y in self.arr
```

```
c = MyClass()
```

```
print("Равно" if c == 50 else "Не равно") # Выведет: Равно
```

```
print("Равно" if c == 51 else "Не равно") # Выведет: Не равно
```

```
print("Есть" if 5 in c else "Нет")    # Выведет: Есть
```

Результат работы приложения:

Равно

Не равно

Есть

Статические методы и методы класса

Внутри класса можно создать метод, который будет доступен без создания экземпляра класса (статический метод). Для этого перед определением метода внутри класса следует указать декоратор **@staticmethod**. Вызов статического метода без создания экземпляра класса осуществляется следующим образом:

<Название класса>.<Название метода>(<Параметры>)

Кроме того, можно вызвать статический метод через экземпляр класса:

<Экземпляр класса>.<Название метода> (<Параметры>)

Приведем пример использования статических методов:

```
class MyClass:
    @staticmethod
    def func1(x, y):          # Статический метод
        return x + y
    def func2(self, x, y):    # Обычный метод в классе
        return x + y
    def func3(self, x, y):
        return MyClass.func1(x, y) # Вызов из метода класса
print(MyClass.func1(10, 20)) # Вызываем статический метод
c = MyClass()
print(c.func2(15, 6))        # Вызываем метод класса
print(c.func1(50, 12))       # Вызываем статический метод
                             # через экземпляр класса
print(c.func3(23, 5))        # Вызываем статический метод
                             # внутри класса
```

Результат работы приложения:

30

21

62

28

Обратите внимание на то, что в определении статического метода нет параметра **self**. Это означает, что внутри статического метода нет доступа к атрибутам и методам экземпляра класса.

Методы класса создаются с помощью декоратора **@classmethod**. В качестве первого параметра в метод класса передается ссылка на класс, а не на экземпляр класса. Вызов метода класса осуществляется следующим образом:

<Название класса>.<Название метода>(<Параметры>)

Кроме того, можно вызвать метод класса через экземпляр класса:

<Экземпляр класса>.<Название метода> (<Параметры>)

Пример использования методов класса:

```
class MyClass:
    @classmethod
    def func(cls, x): # Метод класса
        print(cls, x)
MyClass.func(10)    # Вызываем метод через название класса
c = MyClass()
c.func(50)          # Вызываем метод класса через экземпляр
```

Результат работы приложения:

```
<class '__main__.MyClass'> 10
<class '__main__.MyClass'> 50
```

Абстрактные методы

Абстрактные методы содержат только определение метода без реализации. Предполагается, что класс-потомок должен переопределить метод и реализовать его функциональность. Чтобы такое предположение сделать более очевидным, часто внутри абстрактного метода возбуждают исключение:

```
class Class1:
    def func(self, x): # Абстрактный метод
        # Возбуждаем исключение с помощью raise
        raise NotImplementedError("Необходимо переопределить метод")
class Class2(Class1): # Наследуем абстрактный метод
    def func(self, x): # Переопределяем метод
        print(x)
class Class3(Class1): # Класс не переопределяет метод
    pass
c2 = Class2()
c2.func(50)          # Выведет: 50
c3 = Class3()
try:                 # Перехватываем исключения
    c3.func(50)       # Ошибка. Метод func() не переопределен
except NotImplementedError as msg:
    print(msg)        # Выведет: Необходимо переопределить метод
```

Результат работы приложения:

```
50
Необходимо переопределить метод
```

В состав стандартной библиотеки входит модуль **abc**. В этом модуле определен декоратор **@abstractmethod**, который позволяет указать, что метод, перед которым расположен декоратор, является абстрактным. При попытке создать экземпляр класса-потомка, в котором не переопределен абстрактный метод, возбуждается исключение **TypeError**. Рассмотрим использование декоратора **@abstractmethod** на примере:

```
from abc import ABCMeta, abstractmethod
```



```

class Class1(metaclass=ABCMeta):
    @abstractmethod
    def func(self, x): # Абстрактный метод
        pass
class Class2(Class1): # Наследуем абстрактный метод
    def func(self, x): # Переопределяем метод
        print(x)
class Class3(Class1): # Класс не переопределяет метод
    pass
c2 = Class2()
c2.func(50)          # Выведет: 50
try:
    c3 = Class3()     # Ошибка. Метод func не переопределен
    c3.func(50)
except TypeError as msg:
    print(msg)        # Can't instantiate abstract class Class3
                      # with abstract methods func

```

Результат работы приложения:

50

Can't instantiate abstract class Class3 with abstract methods func

Имеется возможность создания абстрактных статических методов и абстрактных методов класса, для чего необходимые декораторы указываются одновременно, друг за другом. Для примера объявим класс с абстрактными статическим методом и методом класса:

```

from abc import ABCMeta, abstractmethod
class MyClass(metaclass=ABCMeta):
    @staticmethod
    @abstractmethod
    def static_func(self, x): # Абстрактный статический метод
        pass
    @classmethod
    @abstractmethod
    def class_func(self, x): # Абстрактный метод класса
        pass

```

Ограничение доступа к идентификаторам внутри класса

Все идентификаторы внутри класса в языке **Python** являются открытыми, т. е. доступны для непосредственного изменения. Для имитации частных идентификаторов можно воспользоваться методами `__getattr__()`, `__getattribute__()` и `__setattr__()`, которые перехватывают обращения к атрибутам класса. Кроме того, можно воспользоваться идентификаторами, названия которых начинаются с двух символов подчеркивания. Такие идентификаторы называются **псевдочастными**. Псевдочастные идентификаторы доступны лишь внутри класса, но никак не вне его. Тем не менее, изменить идентификатор через экземпляр класса

все равно можно, зная, каким образом искажается название идентификатора. Например, идентификатор **__privateVar** внутри класса **Class1** будет доступен по имени **_Class1__privateVar**. Как можно видеть, здесь перед идентификатором добавляется название класса с предваряющим символом подчеркивания. Приведем пример использования псевдочастных идентификаторов:

```
class MyClass:
    def __init__(self, x):
        self.__privateVar = x
    def set_var(self, x):    # Изменение значения
        self.__privateVar = x
    def get_var(self):      # Получение значения
        return self.__privateVar
c = MyClass(10)            # Создаем экземпляр класса
print(c.get_var())         # Выведет: 10
c.set_var(20)              # Изменяем значение
print(c.get_var())         # Выведет: 20
try:                       # Перехватываем ошибки
    print(c.__privateVar)  # Ошибка!!!
except AttributeError as msg:
    print(msg)             # Выведет: 'MyClass1 object has
                           # no attribute '__privateVar'
c._MyClass__privateVar = 50 # Значение псевдочастных атрибутов
                           # все равно можно изменить
print(c.get_var())        # Выведет: 50
Результат работы приложения:
10
20
'MyClass' object has no attribute '__privateVar'
50
```

Можно также ограничить перечень атрибутов, разрешенных для экземпляров класса. Для этого разрешенные атрибуты перечисляются внутри класса в атрибуте **__slots__**. В качестве значения атрибуту можно присвоить строку или список строк с названиями идентификаторов. Если производится попытка обращения к атрибуту, не перечисленному в **__slots__**, то возбуждается исключение **AttributeError**:

```
class MyClass:
    __slots__ = ["x", "y"]
    def __init__(self, a, b):
        self.x, self.y = a, b
c = MyClass(1, 2)
print(c.x, c.y)    # Выведет: 1 2
c.x, c.y = 10, 20 # Изменяем значения атрибутов
print(c.x, c.y)    # Выведет: 10 20
try:               # Перехватываем исключения
    c.z = 50       # Атрибут z не указан в __slots__,
```

```

# поэтому возбуждается исключение
except AttributeError as msg:
    print(msg)      # 'MyClass' object has no attribute 'z'

```

Результат работы приложения:

```

1 2
10 20
'MyClass' object has no attribute 'z'

```

Инкапсуляция

Скрытие внутреннего устройства объектов называется **инкапсуляцией** и является одним из основных принципов ООП. Такой подход позволяет обезопасить внутренние данные (поля) объекта от изменений (возможно, разрушительных) со стороны других объектов; проверять корректность данных, поступающих от других объектов, повышая тем самым надежность программного кода; передавать внутреннюю структуру и код объекта любым способом, не меняя его внешние характеристики (интерфейс), и при этом никакой переделки других объектов не требуется.

По умолчанию все атрибуты и методы объекта являются **открытыми**. Соответственно, из основного кода программы можно использовать значения атрибутов и вызывать методы. Но для реализации инкапсуляции можно сделать атрибуты и методы класса **закрытыми**.

Предположим, при создании класса **Student** мы хотим оставить открытыми атрибуты, отвечающие за имя студента и город, в котором он проживает. Информацию о возрасте студента мы хотим сделать недоступной. Для того чтобы сделать подобный атрибут закрытым, мы начнем запись его в программе с двух нижних подчеркиваний, например, **self.__vozs**. В следующем коде показано, что доступ к закрытому атрибуту внутри объявления класса может быть осуществлен очень просто. Результатом выполнения программы станет вывод сообщения: «Здравствуйте, я – студент Сергей из Москвы 17».

```

class Student:
    def __init__(self, name, city, vozs):
        self.name=name #Открытые атрибуты
        self.city=city
        self.__vozs=vozs #Закрытый атрибут
    def message(self):
        print("Здравствуйте, я-студент", self.name, "из", self.city, self.__vozs)
st1=Student("Сергей", "Москвы", "17")
st1.message()

```

Продемонстрировать ситуацию, когда закрытие атрибута приводит к ограничению доступа к нему из основного кода, можно на следующем примере. Попытка обратиться к атрибуту **vozs** вне объявления класса **Student** (т. е. просто из командной строки) приводит к ошибке, в которой сообщается о том, что объект **Student** не имеет атрибута **vozs** (рис. 159). Такая же ошибочная ситуация

возникнет даже в том случае, если указать атрибут с двумя нижними подчеркиваниями, например, `print(st1.__vozt)`.

Итак, рассказав о методах создания закрытых атрибутов и способах доступа к ним, можно перейти к разговору о создании **закрытых методов**. Для этого дополним код методом `__talk()`, с помощью которого выведем сообщение: "Начнем урок".

```
def __talk(self):
    print("Начнем урок")
```

Вызовем созданный метод из метода `message()`, созданного ранее, оператором `self.__talk()`.

```
class Student:
    def __init__(self, name, city, vozt):
        self.name=name #Открытые атрибуты
        self.city=city
        self.__vozt=vozt #Закрытый атрибут
    def __talk(self): #Закрытый метод
        print("Начнем урок")
    def message(self):
        print("Здравствуйте, я - студент", self.name, "из", self.city, self.__vozt)
        self.__talk()
st1 = Student("Сергей", "Москвы", "17")
st1.message()
```

После создания закрытого метода можно продемонстрировать ситуацию, когда из пользовательского кода доступ к такому методу не может быть осуществлен. В частности, обращение к классу **Student** с указанием метода `__talk()` вызовет исключение **AttributeError** с сообщением об отсутствии соответствующего атрибута.

Тем не менее, существует возможность вызвать закрытый метод (или получить значение скрытого атрибута), обратившись к имени класса с помощью следующего синтаксиса:

`__ИмяКласса__ИмяАтрибута`

Давайте получим доступ к закрытому атрибуту в нашей программе, написав оператор `print(st1._Student__vozt)`, и вызовем закрытый метод, оператором `st1._Student__talk()`.

После работы над созданием закрытых атрибутов и методов может возникнуть естественный вопрос: «А зачем их делать закрытыми, если доступ к ним все равно имеется?» Ответ можно дать такой: во-первых, случайно воспользоваться закрытым атрибутом или методом не удастся, а, во-вторых, инкапсуляция, т. е. ограничение доступа к методам или переменным, в Python существует на уровне договоренности между программистами, поэтому не стоит стремиться закрывать абсолютно все методы и атрибуты созданного класса.

Пример решения задачи на языке программирования Python

Задача. Создайте класс Time (Время), который предназначен для хранения времени в минутах. В этом классе реализуйте перегрузку операторов сложения и вычитания так, чтобы можно было выполнять операции между двумя объектами времени. При сложении двух объектов времени их значения должны суммироваться, а при вычитании — находиться разность между ними. Результат выполнения операций должен выводиться в виде общего количества минут.

Решение задачи:

```
class Time:
    def __init__(self, minutes):
        self.minutes = minutes
    def __add__(self, other):
        return Time(self.minutes + other.minutes)
    def __sub__(self, other):
        return Time(self.minutes - other.minutes)
    def __str__(self):
        return f"{self.minutes} минут"

# Пример использования
t1 = Time(30)
t2 = Time(20)
print("t1 =", t1)
print("t2 =", t2)
print("Сложение:", t1 + t2)
print("Вычитание:", t1 - t2)
```

Свойства класса

Внутри класса можно создать идентификатор, через который в дальнейшем будут производиться операции получения и изменения значения какого-либо атрибута, а также его удаления. Создается такой идентификатор с помощью функции **property()**. Формат функции:

```
<Свойство> = property(<Чтение>[, <Запись>[, <Удаление>
    [, <Строка документирования>]])
```

В первых трех параметрах указывается ссылка на соответствующий метод класса. При попытке получить значение будет вызван метод, указанный в первом параметре. При операции присваивания значения будет вызван метод, указанный во втором параметре, - этот метод должен принимать один параметр. В случае удаления атрибута вызывается метод, указанный в третьем параметре. Если в качестве какого-либо параметра задано значение None, то это означает, что соответствующий метод не поддерживается. Рассмотрим свойства класса на примере:

```
class MyClass:
    def __init__(self, value):
        self.__var = value
    def get_var(self):      # Чтение
        return self.__var
```

```

def set_var(self, value): # Запись
    self.__var = value
def del_var(self):      # Удаление
    del self.__var
v = property(get_var, set_var, del_var, "Строка документирования")
c = MyClass(5)
c.v = 35 # Вызывается метод set_var()
print(c.v)      # Вызывается метод get_var()
del c.v         # Вызывается метод del_var()

```

Результат работы приложения:

35

Python поддерживает альтернативный метод определения свойств - с помощью методов **property()**, **setter()** и **deleter()**, которые используются в декораторах. Пример их использования приведен ниже:

```

class MyClass:
    def __init__(self, value):
        self.__var = value
    @property
    def v(self):      # Чтение
        return self.__var
    @v.setter
    def v(self, value): # Запись
        self.__var = value
    @v.deleter
    def v(self):      # Удаление
        del self.__var
c = MyClass(5)
c.v = 35 # Чтение
print(c.v)      # Запись
del c.v         # Удаление

```

Результат работы приложения:

35

Имеется возможность определить абстрактное свойство - в этом случае все реализующие его методы должны быть переопределены в подклассе. Выполняется это с помощью декоратора **@abstractmethod** из модуля **abc**. Пример определения абстрактного свойства показан ниже:

```

from abc import ABCMeta, abstractmethod
class MyClass1 (metaclass=ABCMeta):
    def __init__(self, value):
        self.__var = value
    @property
    @abstractmethod
    def v(self):      # Чтение
        return self.__var
    @v.setter

```

```

@abstractmethod
def v(self, value): # Запись
    self.__var = value
@v.deleter
@abstractmethod
def v(self): # Удаление
    del self.__var

```

Создание свойств

Получить доступ к атрибуту, в том числе закрытому, в Python можно с помощью **свойств**. Чтобы создать свойство, следует написать метод, который вернет значение, однако перед тем, как метод будет объявлен, надо написать декоратор **@property**. Продемонстрируем создание свойства на примере, для чего напишем метод **vozs**, с помощью которого получим доступ к закрытому атрибуту **__vozs**.

Метод **vozs** ниже предваряем декоратором **@property**, а имя свойства будет совпадать с именем метода.

```

class Student:
    def __init__(self, name, city, vozs):
        self.name = name
        self.city = city
        self._vozs = vozs # используйте приватное поле с одним
подчеркиванием
    @property
    def vozs(self):
        return self._vozs # возвращайте приватное поле
    def message(self):
        print("Здравствуйте, я студент", self.name, "из", self.city, "возраст",
self.vozs)
stl = Student("Сергей", "Москва", "17")
stl.message()

```

Теперь через свойство **vozs** получить доступ к значению атрибута внутри или вне объявления класса. Непосредственно в коде оператором **self.vozs** вызывается метод **vozs**, который вернет атрибут **__vozs**. Происходит обращение к методу **vozs**, которое возвратит значение **__vozs**, т. е. число 17.

Декораторы классов в Python

В языке **Python**, помимо декораторов функций, поддерживаются также декораторы классов, которые позволяют изменить поведение самих классов. В качестве параметра декоратор принимает ссылку на объект класса, поведение которого необходимо изменить, и должен возвращать ссылку на тот же класс или какой-либо другой. Пример декорирования класса показан ниже:

```

def deco(C): # Принимает объект класса

```

```

    print("Внутри декоратора") # Производит какие-то действия
    return C                    # Возвращает объект класса
@deco
class MyClass:
    def __init__(self, value):
        self.v = value
c = MyClass(5)
print(c.v)

```

Результат работы приложения:
Внутри декоратора
5

Декораторы функций в Python

Для того, чтобы понять, как работают декораторы, в первую очередь следует вспомнить, что функции в python являются объектами, соответственно, их можно возвращать из другой функции или передавать в качестве аргумента.

Декораторы в Python представляют функцию, которая в качестве параметра получает функцию и в качестве результата также возвращает функцию. Декораторы позволяют модифицировать выполняемую функцию, значения ее параметров и ее результат без изменения исходного кода этой функции.

Рассмотрим простейший пример:

определение функции декоратора

```

def select(input_func):
    def output_func():      # определяем функцию, которая будет выполняться
                             # вместо оригинальной
        print("*****")   # перед выводом оригинальной функции
                             # выводим всякую звездочки
        input_func()       # вызов оригинальной функции
        print("*****")   # после вывода оригинальной функции
                             # выводим всякую звездочки
    return output_func     # возвращаем новую функцию
# определение оригинальной функции
@select                    # применение декоратора select
def hello():
    print("Hello METANIT.COM")
# вызов оригинальной функции
hello()

```

Вначале определяется собственно функция декоратора, которая в данном случае называется select(). В качестве параметра декоратор получает функцию (в данном случае параметр input_func), к которой этот декоратор будет применяться:

```

def select(input_func):
    def output_func():      # определяем функцию, которая будет выполняться
                             # вместо оригинальной

```



```

        print("*****") # перед выводом оригинальной функции
выводим всякую звездочки
        input_func()      # вызов оригинальной функции
        print("*****") # после вывода оригинальной функции
выводим всякую звездочки
        return output_func # возвращаем новую функцию

```

Результатом декоратора в данном случае является локальная функция `output_func`, в которой вызывается входная функция `input_func`. Для простоты здесь перед и после вызова `input_func` для красоты просто выводим набор символов `"*"`.

Далее определяется стандартная функция, к которой применяется декоратор - в данном случае это функция `hello`, которая просто выводит на консоль некоторую строку:

```

@select # применение декоратора select
def hello():
    print("Hello METANIT.COM")

```

Для применения декоратора перед определением функции указывается символ `@`, после которого идет имя декоратора. То есть в данном случае к функции `hello()` применяется декоратор `select()`.

Далее вызываем обычную функцию:

```
hello()
```

Поскольку к этой функции применяется декоратор `select`, то в результате функция `hello` передается в декоратор `select()` в качестве параметра `input_func`. И поскольку декоратор возвращает новую функцию - `output_func`, то фактически в данном случае будет выполняться именно эта функция `output_func()`

В итоге мы получим следующий консольный вывод:

```

*****
Hello METANIT.COM
*****

```

Получение параметров функции в декораторе

Декоратор может перехватывать передаваемые в функцию аргументы:

```

# определение функции декоратора
def check(input_func):
    def output_func(*args): # через *args получаем значения параметров
оригинальной функции
        input_func(*args) # вызов оригинальной функции
        return output_func # возвращаем новую функцию
    # определение оригинальной функции
    @check
    def print_person(name, age):
        print(f"Name: {name} Age: {age}")
    # вызов оригинальной функции
    print_person("Tom", 38)

```

Здесь функция `print_person()` принимает два параметра: `name` (имя) и `age` (возраст). К этой функции применяется декоратор `check()`

В декораторе `check` возвращается локальная функция `output_func()`, которая принимает некоторый набор значений в виде параметра `*args` - это те значения, которые передаются в оригинальную функцию, к которой применяется декоратор. То есть в данном случае `*args` будет содержать значения параметров `name` и `age`.

```
def check(input_func):
    def output_func(*args):      # через *args получаем значения параметров
    функции input_func
```

Здесь просто передаем эти значения в оригинальную функцию:

```
input_func(*args)
```

В итоге в данном получим следующий консольный вывод:

```
Name: Tom Age: 38
```

Но что, если в функцию `print_person` будет передано какое-то недопустимое значение, например, отрицательный возраст? Одним из преимуществ декораторов как раз является то, что мы можем проверить и при необходимости модифицировать значения параметров. Например:

```
# определение функции декоратора
def check(input_func):
    def output_func(*args):
        name = args[0]
        age = args[1]      # получаем значение второго параметра
        if age < 0: age = 1  # если возраст отрицательный, изменяем его
                             значение на 1
```

```
        input_func(name, age) # передаем функции значения для параметров
    return output_func
```

```
# определение оригинальной функции
```

```
@check
```

```
def print_person(name, age):
    print(f"Name: {name} Age: {age}")
```

```
# вызов оригинальной функции
```

```
print_person("Tom", 38)
```

```
print_person("Bob", -5)
```

`args` фактически представляет набор значений, и, используя индексы, мы можем получить значения параметров по позиции и что-то с ними сделать. Так, здесь, если значение возраста меньше 0, то устанавливаем 1. Затем передаем эти значения в вызов функции. В итоге здесь получим следующий вывод:

```
Name: Tom Age: 38
```

```
Name: Bob Age: 1
```

Получение результата функции

Подобным образом можно получить результат функции и при необходимости изменить его:

```
# определение функции декоратора
```

```

def check(input_func):
    def output_func(*args):
        result = input_func(*args)    # передаем функции значения для
        # параметров
        if result < 0: result = 0    # если результат функции меньше нуля, то
        # возвращаем 0
        return result
    return output_func
# определение оригинальной функции
@check
def sum(a, b):
    return a + b
# вызов оригинальной функции
result1 = sum(10, 20)
print(result1)    # 30
result2 = sum(10, -20)
print(result2)    # 0

```

Здесь определена функция `sum()`, которая возвращает сумму чисел. В декораторе `check` проверяем результат функции и для простоты, если он меньше нуля, то возвращаем 0.

Консольный вывод программы:

```

30
0

```

Пример решения задачи на языке программирования Python

Задача. Создайте декоратор класса, который автоматически добавляет к любому классу атрибут `created = True`. Данный атрибут должен появляться у всех объектов класса после его применения. Проверьте работу декоратора на любом примере класса.

Решение задачи:

```

def add_created_attribute(cls):
    cls.created = True
    return cls
@add_created_attribute
class Student:
    def __init__(self, name):
        self.name = name
@add_created_attribute
class Product:
    def __init__(self, title):
        self.title = title
# Проверка
s = Student("Alex")
p = Product("Laptop")

```

```
print(s.created) # True  
print(p.created) # True
```